Programalama Laboratuvarı 2. Proje Ödevi

Kağan Can Kaba – 200201049

Özet: Bu çalışma, Nesneye Yönelik Programlama (NYP) ilkelerine dayalı bir Kule Savunma (Tower Defense) oyununun tasarım ve uygulama süreçlerini sunmaktadır. Çalışmanın temel amacı; Kalıtım, Polimorfizm, Kapsülleme ve Soyutlama gibi temel yazılım mimarisi prensiplerini kullanarak, Unity oyun motoru ve C# programlama dili üzerinde bir oyun simülasyonu oluşturmaktır. Proje, özgün bir "Cyberworld" (Siber Dünya) teması üzerine kurgulanmış olup; Electric Tower, Nuclear Tower ve Slime Tower savunma birimleri ile Cyborg, Cybertank ve Cybertruck düşman sınıflarının etkileşimini konu almaktadır. Geliştirilen sistemde, oyunun mantıksal hesaplamalarını yapan simülasyon motoru ile kullanıcı etkileşimini sağlayan Grafiksel Kullanıcı Arayüzü (GUI) eş zamanlı olarak çalıştırılmıştır. Sonuç olarak, programlama bilgisinin bir oyun geliştirme sürecine aktarıldığı ve olay günlüklerinin (log) eksiksiz tutulduğu bir yazılım ortaya konmuştur.

Anahtar Kelimeler: Kule Savunma Oyunu, Nesneye Yönelik Programlama, Unity, C#, Oyun Geliştirme, Simülasyon, Siber Dünya

I. GİRİŞ

- Bu proje; teorik NYP bilgilerinin, gerçek zamanlı bir Kule Savunma (Tower Defense) simülasyonu üzerinde pratiğe dökülmesini amaçlamaktadır.
- Projenin temel hedefi; Kalıtım (Inheritance), Polimorfizm (Polymorphism), Kapsülleme (Encapsulation) ve Soyutlama (Abstraction) ilkelerini kullanarak, arka planda çalışan matematiksel bir simülasyon motoru ile ön yüzde çalışan interaktif bir Grafiksel Kullanıcı Arayüzünü (GUI) entegre etmektir. Bu amaç doğrultusunda, görselleştirme ve fizik motoru yetenekleri nedeniyle Unity Oyun Motoru ve programlama dili olarak C# tercih edilmiştir..
- Geliştirilen oyun, "Cyberworld" (Siber Dünya) konsepti üzerine kurgulanmıştır; düşmanlar (Cyborg, Cybertank, Cybertruck) ve savunma kuleleri (Electric, Nuclear, Slime Tower) teknolojik bir evrenin parçası olarak modellenmiştir. Proje, düşman dalgalarının (Wave Management) yönetimi, hasar hesaplamaları ve anlık olay günlüğü (Log) tutma gibi teknik isterleri karşılayan bir simülasyon yazılımıdır.
- Bu raporda; projenin geliştirilme sürecinde izlenen yöntemler, oluşturulan sınıf diyagramları (UML), Unity ortamındaki sahne (Scene) yönetimleri (StartScene, SampleScene, GameOverScene) ve elde edilen simülasyon sonuçları detaylı bir şekilde ele alınacaktır.

II. YÖNTEM

Bu projede, Unity oyun motorunun bileşen tabanlı (Component-Based) mimarisi ile Nesneye Yönelik Programlama (NYP) prensipleri hibrit bir yapıda kullanılmıştır. Simülasyon motoru ve Grafiksel Kullanıcı Arayüzü (GUI) arasındaki veri akışı, merkezi bir yönetim sınıfı ve olay tabanlı tetikleyicilerle sağlanmıştır.

A. Sınıf Mimarisi ve UML Tasarımı

A.1) Merkezi Yönetici

- Oyunun genel durumunu yöneten **BaseScript** sınıfı, Singleton tasarım kullanılarak oluşturulmuştur. Bu yapı, sahnedeki diğer tüm nesnelerin (düşmanlar, kuleler, butonlar) oyuncunun parasına, can değerine ve log sistemine **BaseScript.Instance** üzerinden doğrudan ve güvenli bir şekilde erişmesini sağlar.

A.2) Düşman Sınıf Hiyerarşisi (Inheritance & Polymorphism)

- **Ana Sınıf (Enemy):** Tüm düşmanlar için ortak olan currentHealth, velocity ve prize (ödül) değişkenlerini ve hareket mantığını (UpdatePosition) barındırır. Ayrıca hasar alındığında görsel geri bildirim veren (renk değişimi) Coroutine yapıları bu sınıfta tanımlanmıştır.
- **Türemiş Sınıflar:** CyberTank, CyberTruck ve Cyborg sınıfları Enemy sınıfından türetilmiştir. Özellikle CyberTank sınıfı, zırh mekaniğini devreye sokmak için TakeElectricDamage gibi hasar alma metodlarını override ederek gelen hasarı zırh formülüne göre azaltır. Cyborg sınıfı ise farklı hız ve can değerleriyle yapılandırılmıştır.

A.3) Kule Sınıf Hiyerarşisi (Abstraction)

- **Ana Sınıf (FiringTower):** Kulelerin temel işlevi olan "en yakın düşmanı bulma" (UpdateNearestEnemy) ve "mesafe hesaplama" algoritmaları bu soyutlamada tutulur.
- **Özelleşmiş Kuleler:** ElectricTowerScript, NuclearTowerScript ve SlimeTowerScript; ana sınıftaki Shoot metodunu override ederek kendi saldırı tiplerini uygular. Örneğin SlimeTower, hasarın yanı sıra düşmanı yavaşlatma (SlowDown) etkisini de tetikler.

B. GUI ile Simülasyon Motorunun Haberleşmesi

B.1) GUI'den Simülasyona (Input Handling)

- Kullanıcı arayüzündeki butonlar (TowerButtonScript), oyuncunun mevcut parasını BaseScript.Instance.playerMoney üzerinden anlık kontrol eder. Para yeterliyse buton aktifleşir. Kullanıcı bir kule seçtiğinde, BaseScript üzerindeki selectedTower değişkeni güncellenir. Ardından haritadaki kule alanlarına (TowerAreaButton) tıklandığında, simülasyon motoru devreye girer ve Instantiate metodu ile ilgili kule prefab'ı oyun dünyasında oluşturulur.

B.2) Simülasyondan GUI'ye (Data Binding): Oyun döngüsü (Update), simülasyon verilerini her karede (frame) arayüze yansıtır.

- **Sağlık Barları:** Her düşman ve ana üs, kendi currentHealth değerini maksimum can değerine oranlayarak sağlık barı görselinin (rectTransform veya localScale) boyutunu günceller. Bu işlem Enemy sınıfındaki UpdateHealth metodunda matematiksel olarak hesaplanıp görselleştirilir.
- **Oyun Durumu:** BaseScript, oyuncunun canı 0'a düştüğünde veya dalgalar başarıyla tamamlandığında SceneManager aracılığıyla "GameOverScene" sahnesini yükleyerek arayüzü tamamen değiştirir.

B.3) Algoritmik İşleyiş (Pathfinding ve Spawner)

- **Hareket Mantığı:** Düşmanlar, Vector3.right vektörü kullanılarak doğrusal bir yörüngede sağa doğru hareket ettirilmiştir**.**
- **Dalga Yönetimi:** EnemySpawner sınıfı, IEnumerator (Coroutine) yapısını kullanarak düşmanları belirli zaman aralıklarıyla (spawnInterval) asenkron olarak oluşturur. BaseScript seviye (level) bilgisini artırdıkça, spawner'a gönderilen düşman sayısı dinamik olarak güncellenir.

III. OYUN KONSEPTİ

Bu projede, klasik kule savunma mekanikleri fütüristik bir distopyayı konu alan "Cyberworld" (Siber Dünya) teması üzerine kurgulanmıştır. Seçilen bu konsept; oyunun görsel tasarımından sınıf isimlendirmelerine, hasar mekaniklerinden düşman davranışlarına kadar projenin tüm katmanlarına tutarlı bir şekilde entegre edilmiştir.

A. Konsept Seçimi ve Gerekçesi

Proje için seçilen "Cyberworld" (Siber Dünya) teması, fütüristik bir distopyada geçmektedir. Senaryoya göre, kontrolden çıkan yapay zeka destekli robotik ordular, insanlığın son enerji kaynağını (Base/Ana Üs) ele geçirmek için saldırıya geçmiştir.
Bu konseptin seçilme nedenleri şunlardır:

A.1) Görsel Çeşitlilik

- "Cyberworld" teması, lazerler, patlamalar ve neon renk paletleri (yeşil, kırmızı, mavi) kullanılarak oyunun görsel geri bildirimlerinin (visual feedback) oyuncuya daha net aktarılmasını sağlamıştır. Örneğin, kod içerisinde tanımlanan ShowElectricDamage (sarı tonlama) ve ShowNuclearDamage (mavi tonlama) fonksiyonları bu tema ile uyumludur.

A.2) Mantıksal Tutarlılık

- Kule Savunma oyunlarındaki "dalga" (wave) mantığı, "robot istilası" senaryosu ile birebir örtüşmektedir. Düşmanların duygusuz makine yığınları olarak modellenmesi, oyunun mekanik yapısına gerçekçilik katmıştır.

B. İsimlendirme Stratejisi ve Dönüşüm Tablosu

Proje isterlerinde belirtilen standart sınıf isimleri, seçilen konsepte uygun olarak yeniden adlandırılmıştır. Bu isimlendirme stratejisi, kodun okunabilirliğini artırmış ve oyun evreninin bütünlüğünü sağlamıştır.

C. Konseptin Oyun Mekaniklerine Yansıması

Seçilen tema, sadece görsel bir kaplama olmanın ötesinde, kodun çalışma mantığını (Business Logic) da şekillendirmiştir:

C.1) Zırh ve Hasar Mekaniği

- CyberTank sınıfında tanımlanan zırh mekaniği, hasar azaltan bir zırh mantığıyla çalışmaktadır. Kod içerisindeki GetNetDamage fonksiyonu, **1f - (armor / (armor + 100f))** formülüyle gelen hasarı hesaplar. Bu, tankın ağır metal gövdesinin gelen saldırılara karşı direncini temsil eder.

C.2) Kimyasal Yavaşlatma (Slime)

- SlimeTower, düşmanlara fiziksel hasar vermenin yanı sıra, SlimeTowerScript içerisinde tetiklenen logic ile düşmanların hız değişkenini (velocity) azaltır. TakeSlimeDamage metodu, düşmanın rengini değiştirerek (Visual Feedback) üzerine yapışkan madde bulaştığını gösterir ve hareket hızını halfVelocity değerine düşürür.

C.3) Nükleer Alan Hasarı

- NuclearTower, bir mermi fırlatmak yerine NuclearBombScript aracılığıyla bir patlama alanı oluşturur. Kodda OnTriggerEnter2D kullanılarak, patlama alanına giren Cyborg haricindeki tüm düşmanlara toplu hasar verilmesi, nükleer bomba mantığıyla kurgulanmıştır.

C.4) Görsel Geri Bildirimler

- Düşmanlar hasar aldığında SpriteRenderer bileşenlerinin renk değiştirmesi (kırmızıya dönme, yeşile dönem), robotların hasar almaları veya slime tarafından yavaşlamaları şeklinde yorumlanmıştır.

IV. SİMÜLASYON

A. "savunma_gunlugu.txt" Çıktıları

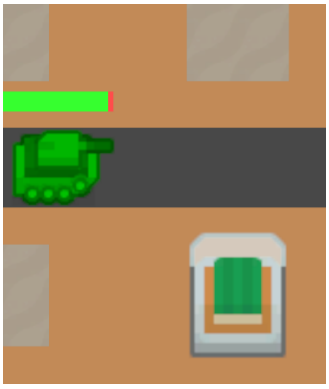
```
--- GAME STARTED: 2.12.2025 22:22:18 ---
[22:22:18] 1. Wave Started.
[22:22:18] Enemy CyberTank has entered the map. HP: 75 Armor: 100
[22:22:19] Player added Electric Tower to (-4.50, -1.60, 0.00) position
[22:22:21] Player added Nuclear Tower to  (-4.50, 1.72, 0.00) position
[22:22:21] Enemy Cyborg has entered the map. HP: 50
[22:22:21] Player added Slime Tower to (-1.50, -1.60, 0.00) position
[22:22:22] Tower: Electric Tower shot nearest enemy: CyberTank(Clone)
[22:22:22] (Damage: 10)
[22:22:22] Armored Enemy's Net Damage: 2,5
[22:22:22] Current Health of CyberTank: 72,5
[22:22:22] Tower: Nuclear Tower shot nearest enemy: CyberTank(Clone)
[22:22:22] (Area damage 20 on (-6.67, 0.00, 0.00))(can't aim Cyborgs)
[22:22:22] Armored Enemy's Net Damage: 10
[22:22:22] Current Health of CyberTank: 62,5
[22:22:23] Tower: Electric Tower shot nearest enemy: CyberTank(Clone)
[22:22:23] (Damage: 10)
[22:22:23] Armored Enemy's Net Damage: 2,5
[22:22:23] Current Health of CyberTank: 60
[22:22:24] Tower: Electric Tower shot nearest enemy: Cyborg(Clone)
[22:22:24] (Damage: 10)
[22:22:24] Current Health of Cyborg: 40
[22:22:24] Enemy CyberTruck has entered the map. HP: 50
[22:22:24] Tower: Slime Tower shot nearest enemy: Cyborg(Clone)
[22:22:24] (Slowed 50% with 15 damage)
[22:22:24] Current Health of Cyborg: 25
[22:22:25] Tower: Electric Tower shot nearest enemy: CyberTank(Clone)
[22:22:25] (Damage: 10)
[22:22:25] Armored Enemy's Net Damage: 2,5
[22:22:25] Current Health of CyberTank: 57,5
[22:22:25] Tower: Nuclear Tower shot nearest enemy: CyberTank(Clone)
[22:22:25] (Area damage 20 on (-4.80, 0.00, 0.00))(can't aim Cyborgs)
[22:22:25] Armored Enemy's Net Damage: 10
[22:22:25] Current Health of CyberTank: 47,5
[22:22:26] Tower: Electric Tower shot nearest enemy: CyberTank(Clone)
[22:22:26] (Damage: 10)
[22:22:26] Armored Enemy's Net Damage: 2,5
[22:22:26] Current Health of CyberTank: 45
[22:22:26] Tower: Slime Tower shot nearest enemy: Cyborg(Clone)
[22:22:26] (Slowed 50% with 15 damage)
[22:22:26] Current Health of Cyborg: 10
[22:22:27] Tower: Electric Tower shot nearest enemy: CyberTank(Clone)
[22:22:27] (Damage: 10)
[22:22:27] Armored Enemy's Net Damage: 2,5
[22:22:27] Current Health of CyberTank: 42,5
```

```
[22:23:12] Tower: Electric Tower shot nearest enemy: CyberTank(Clone)
[22:23:12] (Damage: 10)
[22:23:12] Armored Enemy's Net Damage: 2,5
[22:23:12] Current Health of CyberTank: 0
[22:23:12] CyberTank has been killed. Prize: 20 Total Money: 55
[22:23:12] Game has been Won  Last HP: 75 Last Money: 55
```

B. Canlı Görselleştirmeden Alınan Ekran Görüntüleri

- ***Hasar Alan Cyborg Görüntsüsü***

- ***Slime Tower Tarafından Yavaşlatılmış CyberTank Görüntüsü***



- ***Nuclear Tower Tarafından Verilen Alan Hasarı Görüntüsü***

V. SONUÇ

Bu projede, Nesneye Yönelik Programlama (NYP) ilkelerine tam uyumlu, "Cyberworld" temalı bir Kule Savunma (Tower Defense) oyunu başarıyla geliştirilmiştir.

Projenin en temel hedefi olan mimari tasarımda Kalıtım (Inheritance), Soyutlama (Abstraction), Kapsülleme (Encapsulation) ve Polimorfizm (Polymorphism) ilkeleri etkin bir şekilde kullanılmıştır.

Enemy ve FiringTower temel sınıfları sayesinde kod tekrarı önlenmiş, modüler bir yapı kurulmuştur.

Düşmanların hasar alma (TakeDamage) ve kulelerin ateş etme (Shoot) mekanizmalarında metot ezme (Overriding) kullanılarak polimorfik davranışlar sergilenmiş; böylece her birimin (Örn: CyberTank, SlimeTower) kendine özgü algoritmaları ana yapı bozulmadan sisteme entegre edilmiştir.

Simülasyon ve GUI Entegrasyonu: Unity oyun motorunun bileşen tabanlı yapısı ile C# dili kullanılarak, arka plandaki matematiksel hesaplamalar (hasar formülleri, zırh hesapları) ile ön yüzdeki görselleştirmeler (renk değişimleri, can barları) eş zamanlı (real-time) olarak çalıştırılmıştır. BaseScript sınıfında uygulanan Singleton Tasarım Deseni, oyun verilerinin (para, can, dalga sayısı) sahneler ve nesneler arasında güvenli ve merkezi bir şekilde yönetilmesini sağlamıştır.

Algoritmik Doğruluk ve Kayıt Sistemi: Projenin işlevsel gereksinimlerinden biri olan olay günlüğü tutma işlemi, savunma_gunlugu.txt dosyası üzerinden başarıyla gerçekleştirilmiştir. Log dosyası analizleri, özellikle Zırhlı Düşman (CyberTank) üzerindeki hasar azaltma formüllerinin ve kulelerin hedef önceliklendirme algoritmalarının ("En Yakın Düşman" mantığı) hatasız çalıştığını kanıtlamıştır.

Performans ve Genişletilebilirlik: Düşman üretiminde (Spawner) kullanılan Coroutine yapıları (IEnumerator), ana oyun döngüsünü (Main Thread) bloklamadan akıcı bir oyun deneyimi sunmuştur. Kurulan esnek sınıf hiyerarşisi sayesinde, gelecekte projeye yeni düşman veya kule türlerinin eklenmesi, mevcut kodda köklü değişiklikler yapılmasına gerek kalmadan mümkün hale getirilmiştir.

Sonuç olarak; teorik programlama bilgisinin pratik bir oyun geliştirme sürecine başarıyla aktarıldığı, teknik isterlerin ve tasarım desenlerinin doğru kullanıldığı, görsel ve işlevsel açıdan tamamlanmış bir mühendislik projesi ortaya konmuştur.

VI. YAZAR KATKILARI

Bu proje, Kağan Can Kaba tarafından bireysel çalışma olarak hazırlanmıştır. Projenin yaşam döngüsü boyunca gerçekleştirilen katkılar aşağıda belirtilmiştir:

Mimari Tasarım ve Analiz: Oyunun "Cyberworld" konseptinin kurgulanması, UML sınıf diyagramlarının oluşturulması ve nesneye yönelik programlama (NYP) hiyerarşisinin (Kalıtım, Polimorfizm yapıları) planlanması.

Yazılım Geliştirme (Backend): Simülasyon motorunu oluşturan tüm C# classlarının (BaseScript, Enemy, FiringTower, Spawner vb.) yazılması, Singleton tasarım deseninin uygulanması ve loglama sisteminin entegrasyonu.

Görselleştirme ve Arayüz (Frontend): Unity editörü üzerinde sahne tasarımlarının yapılması, prefab'ların (Kule ve Düşman varlıkları) oluşturulması, animasyon/efektlerin ayarlanması ve kullanıcı arayüzü (Canvas, Butonlar) etkileşimlerinin kodlanması.

VII. KAYNAKÇA

[1] https://refactoring.guru/design-patterns/singleton

[2] https://kenney.nl/assets

[3] https://docs.unity3d.com/Manual/index.html

[4] https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/

[5] https://youtu.be/XtQMytORBmM